EOB Extractor — Integration Contracts

Date: 2026-06-08

This document defines the exact data contracts for all integration boundaries in the EOB Extractor pipeline.

---

1. SQS Event Schemas

1.1 `eob-extractor-ingest` — Inbound Message

Messages are placed on the ingest queue by S3 ObjectCreated event notifications. The queue forwards them to `eob-trigger` as SQS batches (batchSize=1).

SQS record body (stringified JSON — S3 event notification format):

```
{
  "Records": [
    {
      "s3": {
        "bucket": {
          "name": "bucket-specialops"
        },
        "object": {
          "key": "clickup%2F9az3bkxh1%2Fdocument.pdf"
        }
      }
    }
  ]
}
```

- `object.key` is URL-encoded; `eob-trigger` decodes via `decodeURIComponent(...replace(/\+/g, ' '))`
- Only `Records[0]` is consumed per SQS message

SQS Lambda event wrapper:

```
interface SQSEvent {
  Records: Array<{
    messageId: string;
    body: string; // stringified S3EventNotification above
    // ...standard SQS fields
  }>;
}
```

1.2 `eob-extractor-ingest` → SFN Execution Input

`eob-trigger` starts Step Functions with this JSON input:

```
{
  "bucket": "bucket-specialops",
  "key": "clickup/9az3bkxh1/document.pdf",
  "taskId": "9az3bkxh1",
  "correlationId": "01J3XYZ..."
}
```

Field	Type	Description
`bucket`	string	S3 bucket name
key	string	Decoded S3 object key
`taskId`	string	Extracted from key path segment between `clickup/` and the filename
`correlationId`	string	ULID generated by `eob-trigger` for this execution

SFN execution name format: `eob-{taskId}-{correlationId}`

1.3 `eob-extractor-review` — Review Queue Message

Placed by `eob-store-result` when `status === REVIEW_PENDING`:

```
{
  "extractionId": "01J3ABC...",
  "taskId": "9az3bkxh1",
  "correlationId": "01J3XYZ...",
  "confidenceScore": 0.72,
  "status": "REVIEW_PENDING"
}
```

Note (TD-005): This queue has no DLQ. If a downstream consumer fails to process a message after the visibility timeout expires, the message re-enters the queue indefinitely until the 14-day retention period elapses. A DLQ must be added.

---

2. Step Functions — Inter-Lambda Contracts

Each Lambda receives the accumulated state from the previous step. The state object grows with each step.

2.1 ValidatePDF input/output

Input:

```
{
  "bucket": "bucket-specialops",
  "key": "clickup/9az3bkxh1/document.pdf",
  "taskId": "9az3bkxh1",
  "correlationId": "01J3XYZ..."
}
```

Output (added fields):

```
{
  "bucket": "...",
  "key": "...",
  "taskId": "...",
  "correlationId": "...",
  "valid": true,
  "versionId": "abc123"
}
```

Choice state checks `$.valid`. If `false`, execution ends at `PDFInvalid` (Succeed).

2.2 ClassifyEOB input/output

Input: ValidatePDF output (adds `valid`, `versionId`)

Output (added fields):

```
{
  "classification": {
    "document_type": "EOB",
    "is_eob": true,
    "insurer_name": "Blue Cross Blue Shield of Illinois",
    "insurer_identifier": "00710",
    "confidence": 0.97
  },
  "classifyModelId": "us.anthropic.claude-haiku-4-5-20251001-v1:0",
  "isEob": true
}
```

Choice state checks `$.isEob`. If `false`, execution ends at `NotAnEOB` (Succeed).

2.3 ExtractEOB input/output

Input: ClassifyEOB output

Output (added fields):

```
{
  "extraction": "{\"is_eob\":true,\"confidence_score\":0.91,...}",
  "extractModelId": "us.anthropic.claude-sonnet-4-6",
  "processingDurationMs": 4823
}
```

`extraction` is a **JSON string** (not an object) — parsed by `eob-validate-data`.

2.4 ValidateData input/output

Input: ExtractEOB output

Output (added fields):

```
{
  "validatedExtraction": {
    "is_eob": true,
    "confidence_score": 0.91,
    "extraction_notes": null,
    "insurance_name": "Blue Cross Blue Shield of Illinois",
    "insurance_identifier": "00710",
    "address": "300 E Randolph St",
    "city": "Chicago",
    "state": "IL",
    "zip_code": "60601",
    "location_state": "IL",
    "arbitration_phone": "1-800-555-0100",
    "arbitration_fax": null,
    "arbitration_email": "appeals@bcbsil.com"
  },
  "confidenceScore": 0.91,
  "missingFields": [],
  "warnings": [],
  "isValid": true
}
```


Routing: `confidenceScore >= 0.50` → LookupInsurance; otherwise → StoreFailed.

2.5 LookupInsurance input/output

Input: ValidateData output

Output (added fields):


```
{
  "lookupResult": "MATCH",
  "mismatches": [],
  "contactRecord": {
    "Insurance": "*IL - Blue Cross Blue Shield of Illinois",
    "InsuranceName": "Blue Cross Blue Shield of Illinois",
    "LocationState": "IL",
    "Address": "300 E Randolph St",
    "City": "Chicago",
    "State": "IL",
    "ZipCode": "60601",
    "ArbitrationPhone": "1-800-555-0100",
    "ArbitrationFax": "",
    "ArbitrationEmail": "appeals@bcbsil.com"
  }
}
```


`lookupResult` values: - `MATCH` — record exists and all compared fields match - `MISMATCH` — record exists but one or more fields differ; `mismatches` array lists each discrepancy as `"{Field}: extracted="{val}" vs existing="{val}"` - `NEW` — no existing record; new contact created in `Insurance-Arbitration-contacts`

Routing: `MATCH` → StoreExtracted; `MISMATCH` or `NEW` → StoreReviewPending.

2.6 StoreResult input/output

Input: LookupInsurance output (full accumulated state)

Output:

```
{
  "extractionId": "01J3DEF...",
  "taskId": "9az3bkxh1",
  "status": "EXTRACTED",
  "confidenceScore": 0.91
}
```

---

3. Bedrock — Classification Contract

API: `InvokeModelCommand` via `BedrockRuntimeClient`
Model chain: `CLASSIFY_CHAIN` (Haiku → Sonnet)
anthropic_version: `bedrock-2023-05-31`
max_tokens: 8192
temperature: 0

Request (Messages API)

```
{
  "anthropic_version": "bedrock-2023-05-31",
  "max_tokens": 8192,
  "temperature": 0,
  "system": "<EOB_CLASSIFICATION_SYSTEM_PROMPT>",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "document",
          "source": {
            "type": "base64",
            "media_type": "application/pdf",
            "data": "<base64-encoded PDF>"
          }
        },
        {
          "type": "text",
          "text": "Classify this document. Is it an Explanation of Benefits (EOB)? Identify the
insurer if possible."
        }
      ]
    }
  ]
}
```

Expected Response Schema

```
{
  "document_type": "EOB",
  "is_eob": true,
  "insurer_name": "Blue Cross Blue Shield of Illinois",
  "insurer_identifier": "00710",
  "confidence": 0.97
}
```

Field	Type	Description
`document_type`	string	`"EOB"` | `"medical_bill"` | `"provider_statement"` | `"insurance_card"` | `"unknown"`
`is_eob`	boolean	True if EOB document
`insurer_name`	string | null	Insurance company name if identifiable
`insurer_identifier`	string | null	Payer ID or other identifier
`confidence`	number (0–1)	Model confidence in classification

JSON retry: If the response cannot be parsed as JSON, a second turn is sent requesting only valid JSON. If the retry also fails, the error is thrown and SFN routes to `PipelineFailed`.

---

4. Bedrock — Extraction Contract

API: `InvokeModelCommand` via `BedrockRuntimeClient`
Model chain: `EXTRACT_CHAIN` (Sonnet 4.6 → Sonnet 4.0 → Haiku)
anthropic_version: `bedrock-2023-05-31`
max_tokens: 8192
temperature: 0

Request (Messages API)

```
{
  "anthropic_version": "bedrock-2023-05-31",
  "max_tokens": 8192,
  "temperature": 0,
  "system": "<EOB_EXTRACTION_SYSTEM_PROMPT>",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "document",
          "source": {
            "type": "base64",
            "media_type": "application/pdf",
            "data": "<base64-encoded PDF>"
          }
        },
        {
          "type": "text",
          "text": "<buildExtractionUserPrompt(classificationContext)>"
        }
      ]
    }
  ]
}
```

The user prompt includes pre-classification context (insurer name + payer ID) if available, to guide extraction.

Expected Response Schema (Zod-validated)

```
{
  "is_eob": true,
  "confidence_score": 0.91,
  "extraction_notes": null,
  "insurance_name": "Blue Cross Blue Shield of Illinois",
  "insurance_identifier": "00710",
  "address": "300 E Randolph St",
  "city": "Chicago",
  "state": "IL",
  "zip_code": "60601",
  "location_state": "IL",
  "arbitration_phone": "1-800-555-0100",
  "arbitration_fax": null,
  "arbitration_email": "appeals@bcbsil.com"
}
```

All string fields are `string | null`. The pipeline continues even on Zod failure (zeroed extraction, `confidence_score=0.1`).

---

5. Insurance-Arbitration-contacts — Read Contract

Called by: `eob-lookup-insurance`
Operation: DynamoDB `QueryCommand`
Index: `GSI-InsuranceName-LocationState`

```
// KeyConditionExpression
'InsuranceName = :name AND LocationState = :state'
// ExpressionAttributeValues
{ ':name': insuranceName, ':state': locationState }
```

Returns the first matching item or `undefined`. No pagination — assumes insurer+state combination is unique.

---

6. Insurance-Arbitration-contacts — Write Contract

Called by: `eob-lookup-insurance` when no existing record found
Operation: DynamoDB `PutCommand` with conditional expression

```
ConditionExpression: 'attribute_not_exists(Insurance)'
```

Item written:

```
{
  "Insurance": "*{locationState} - {insuranceName}",
  "InsuranceName": "{insuranceName}",
  "LocationState": "{locationState}",
  "Address": "{address or ''}",
  "ArbitrationEmail": "{arbitration_email or ''}",
  "ArbitrationFax": "{arbitration_fax or ''}",
  "ArbitrationPhone": "{arbitration_phone or ''}",
  "City": "{city or ''}",
  "State": "{state or ''}",
  "ZipCode": "{zip_code or ''}"
}
```

`ConditionalCheckFailedException` on concurrent write is silently swallowed (treated as no-op).

---

7. SNS Notification Payloads

7.1 New Insurance Contact (`eob-extractor-review-alerts`)

Subject: `New Insurance Contact: {insurance_name}`

```
{
  "event": "new_insurance_contact",
  "taskId": "9az3bkxh1",
  "correlationId": "01J3XYZ...",
  "insuranceName": "Blue Cross Blue Shield of Illinois",
  "locationState": "IL",
  "s3Key": "clickup/9az3bkxh1/document.pdf",
  "missingFields": [],
  "extractedFields": {
    "address": "300 E Randolph St",
    "city": "Chicago",
    "state": "IL",
    "zipCode": "60601",
    "arbitrationPhone": "1-800-555-0100",
    "arbitrationFax": null,
    "arbitrationEmail": "appeals@bcbsil.com"
  }
}
```

7.2 Insurance Contact Mismatch (`eob-extractor-review-alerts`)

Subject: `Insurance Contact Mismatch: {insurance_name}`

```
{
  "event": "insurance_contact_mismatch",
  "taskId": "9az3bkxh1",
  "correlationId": "01J3XYZ...",
  "insuranceName": "Blue Cross Blue Shield of Illinois",
  "locationState": "IL",
  "existingInsuranceKey": "*IL - Blue Cross Blue Shield of Illinois",
  "s3Key": "clickup/9az3bkxh1/document.pdf",
  "missingFields": [],
  "mismatches": [
    "ArbitrationPhone: extracted=\"1-800-555-0199\" vs existing=\"1-800-555-0100\""
  ]
}
```

---

8. Audit Logger — Log Event Schema

All log lines are single-line JSON emitted to stdout (CloudWatch).

Extraction Complete

```
{
  "timestamp": "2026-06-08T12:00:00.000Z",
  "correlationId": "01J3XYZ...",
  "level": "INFO",
  "event": "eob_extraction_complete",
  "extractionId": "01J3DEF...",
  "modelId": "us.anthropic.claude-sonnet-4-6",
  "status": "EXTRACTED",
  "confidenceScore": 0.91,
  "processingDurationMs": 4823,
  "s3Key": "clickup/9az3bkxh1/*",
  "taskId": "9az3bkxh1"
}
```

Error Event

```
{
  "timestamp": "2026-06-08T12:00:00.000Z",
  "correlationId": "01J3XYZ...",
  "level": "ERROR",
  "event": "eob_extraction_error",
  "errorName": "AllModelsExhaustedException",
  "errorMessage": "All models in fallback chain exhausted...",
  "s3Key": "clickup/9az3bkxh1/*",
  "taskId": "9az3bkxh1"
}
```

All S3 keys in logs are sanitized to `{prefix}/*` — filename is never logged.